

数据集构建结果汇报

概述

相关脚本已经传到了github上的仓库。

`./Di/RAWProcessing`

视频的拍摄使用 iPhone17pro 搭配 black magic 来拍摄，导出格式为npz

`testRAW.py` 检测 npz 文件是否是 RAW 数据

`npz_to_safetensors.py` 将数据对齐团队标准，包括 CFA 排列，位深，大小的对齐，将数据转化成 fp32 safetensors格式存储

`OpenCV_ISP.py` 用于处理一个文件夹的格式为 safetensors 的 RAW 帧，通过 XVID 编码器编码，生成可视化的 avi 视频

最终形成了 12 个场景的数据集，每个场景 240 帧

场景1 ~ 4是同一个背景，场景5 ~ 8是同一个背景，9 ~ 12是同一个背景

每个场景下，RAW 帧命名为 `framei`, $0 \leq i \leq 239$

每个场景配备一个经过 ISP 后的 avi 视频。

1~7场景在夜间拍摄，运动比较剧烈，拖影多。

场景8只有 180 帧，是一段拖影比较少、运动比较平缓的视频

场景9~12是白天低速运动的场景，为了增加运动的多样性，11、12是将视频倒立制成的。

另外配备了一个numpy格式的视频帧和一个safetensors格式的视频帧，可以用于其他测验。

使用小ISO、长曝光拍摄，除了用于运动检测，还可以作为GT。

数据集下载: https://pan.baidu.com/s/1RtSHEuPPTVWb9danhr2o_Q?pwd=c245

检查是否是真RAW

取一个 4x4 的小块，看相邻像素的差异

```
1 sample_block = img[100:104, 100:104]
2 diff_h = np.mean(np.abs(img[:, :-1] - img[:, 1:]))
3 diff_v = np.mean(np.abs(img[:-1, :] - img[1:, :]))
4 if diff_h > 0.01: # 这里的阈值取决于是否归一化，
   float32 归一化后 0.01 已经很大了
5     print("[结论] 大概率是真 Bayer RAW(检测到明显的马赛
   克阶跃特征)")
6 else:
7     print("[结论] 可能不是真 RAW。")
```

数据打包

转化为 fp32 safetensors 格式

Code: npy_to_safetensors.py

集合了画面大小裁剪、CFA转换、位深转换

黑白电平都除以16，cfa换成GBRG，大小换成1920x1080，dtype换成fp32

选取原因

bf16：1位符号 + 8位指数 + 7位尾数。

fp32：1位符号 + 8位指数 + 23位尾数。

由于bf16尾数太小，导致精度不足，在接近白平衡时，可能不能完全表示每个梯度的亮度。导致原本12bit的数据集可能会只有8~9bit

为什么选取 **safetensors**：相比于tiff格式，safetensors格式的读取速度大幅增加，这与其数据排列（连续的二进制排列）有关，在训练时不用让模型“等数据”。

它的构造：

1. **Header (文件头)**：一小段纯文本JSON，描述了里面存了几个张量、叫什么名字、什么形状 (Shape)、什么数据类型 (Dtype) 。
2. **Raw Buffer (原始缓冲区)**：紧跟在文件头后面，是纯粹的、没有任何包装的二进制数据流。

fp32的safetensors读写

```
1 #将一个文件夹的RAW数据全部读入
2 files = [f for f in os.listdir(INPUT_DIR)
3           if f.lower().endswith('.safetensors')]
4 #转化为np数组
5 path = os.path.join(INPUT_DIR, filename)
6 raw_tensor = load_file(path)["raw"]
7 #进行一次数据类型转化，以防万一。如果以后要改成非fp32的存储
  类型，这里就可以派上用场
8 raw_np = raw_tensor.to(torch.float32).numpy()
```

```
1 #转为 torch tensor
2 tensor_data = torch.from_numpy(xxx)
3 #转换为 fp32 精度
4 tensor_fp32 = tensor_data.to(torch.float32)
5 #重新命名并保存
6 new_name =
7 save_path = os.path.join(OUTPUT_DIR, new_name)
8 # safetensors 保存需要一个字典，我们将 key 统一设为
  "raw"
9 save_file({"raw": tensor_fp32}, save_path)
```

4.2k to 2k

经过计算，直接裁剪中间的画面，不会破坏CFA排列

CFA重新排列

```
1 R G R G
2 G B G B
3 R G R G
4 G B G B
```

从RGGB->GBRG，只需要往下移动一行

则起始行是偶数

一个比较有趣的点：G都在对角线的CFA排列是可以通过横向竖向移动（每个方向移动至多一格）来转换的。

(GBRG GRBG BGGR)

以后如果还要拍数据集，尺寸可以拍大点。

位深

除以16，线性变换

数据集说明

如概述中所述

可视化问题

OpenCV_ISP.py

将团队仓库内的ISP模块交给AI，让它进行视频渲染。

主要有以下改动：

1. **黑电平重新标定**。原黑电平为240。按照 metadata.json 中保存的元数据，黑电平为256，经过 16bit 到 12bit 的转化， $256 / 16 = 16$ 。
2. **加入自动白平衡**。之前的代码：

```
1 img[1::2, 0::2] *= self.r_gain # 固定的 R 增益
2 img[0::2, 1::2] *= self.b_gain # 固定的 B 增益
```

改动：

```
1 # 增加了一个类函数，通过第一帧动态计算颜色均值
2 def _compute_auto_white_balance(self, img):
3     # 针对 GBGR 的精确切片：R 在 (1,0)，B 在 (0,1)
4     r_mean = np.mean(img[1::2, 0::2])
5     b_mean = np.mean(img[0::2, 1::2])
6     # 灰度世界假设计算
7     return [g_mean/r_mean, 1.0, g_mean/b_mean]
```

引入了 **AWB (自动白平衡)**，之前的代码在不同场景下（比如室内灯光 vs 室外阳光）会偏黄或偏紫。现在的 AWB 算法能自动校正色彩，保证了渲染出的视频看起来像真实的成品。现在通过第一帧来自动校正色彩，能够让视觉效果更好些。不过我们主要是用来对比效果，影响不大。

3. **从单张到视频**。之前的代码用的是 `matplotlib.pyplot` (`plt.imshow`)，当调用 `plt.show()` 时，程序会停在那里，必须手动关掉窗口，代码才会跑下一帧。现在的脚本采用的是“**视频流编码**”逻辑，把一张张处理好的图塞进一个名为 **VideoWriter** 的容器里。

```

1 # A. 初始化集装箱 (videowriter)
2 # 指定：路径、编码器(XVID)、帧率(FPS)、分辨率(W, H)
3 fourcc = cv2.VideoWriter_fourcc(*'XVID')
4 video_writer = cv2.VideoWriter(OUTPUT_VIDEO,
5     fourcc, FPS, (w, h))
6 # B. 循环流水线
7 for filename in files:
8     raw = load_file(...) # 1. 卸货 (读取)
9     frame_bgr = isp.process_frame(raw) # 2. 加工 (ISP)
10    video_writer.write(frame_bgr) # 3. 装箱 (写入视频流)
11
12 # C. 封箱导出
13 video_writer.release() # 这步没做，视频就打不开

```

4. 加入Gamma校正。

```

1 bgr_gamma = np.power(bgr_f, 1 / 2.2)

```

视频压缩算法（如 XVID）的核心原理是“寻找冗余”：

- **空间冗余**：如果相邻像素颜色差不多，就只存一个值。
- **时间冗余**：如果前后两帧画面差不多，只存变化的差值。

加入物理噪声后：

- 噪声是随机的跳变。原本平滑的色块变成了每个像素都在跳动的乱数。
- 压缩算法无法在噪声中找到规律，被迫记录下每一个像素的随机抖动。
- 噪声越多，信息熵越高，压缩率越低，文件就越大。

总结

产出

1. 形成 **4.2K 到 2K** 处理方案及对应实现。
2. 给出 **CFA 适配方案**的分析结论，并明确推荐做法。
3. 提供将 **16bit 降至 12bit** 的**代码工具**，并撰写一份文档，说明：
 - 位深下降的方法；
 - 下降后如何检测是否正确实现目标。
4. 提供用于数据打包、压缩和保存的**代码工具**，并附上**详细使用说明**。

1. 4.2k到2k：切割正中间的部分
2. CFA适配：切割后的画面向下移动一行，正好变成GBRG
3. 16bit 降成 12bit：线性变换，所有数据除以 16。验证方法：选取一个区块，输出转换前后的像素值。
4. 将数据保存为位深12bit的fp32 safetensors文件，并且将文件重新命名，收入一个统一的文件夹。

以上均可在`numpy_to_sftensor.py`中实现

可视化脚本，如 `OpenCV_ISP.py`，可以将一个文件夹的 safetensors 文件按顺序排列，从而转化为视频。

红叉问题，水印好像无法去除。